Phase 6

Multiply, Divide and Floating Point

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Extend a pipelined RV32I CPU with the RV32M multiply and divide instructions, building the multiplier and divider yourself out of adds, shifts and subtracts.
- Represent, round and compute single-precision floating point numbers following IEEE 754, and implement the RV32F instructions in a floating-point unit.
- Integrate multi-cycle functional units into a pipelined, cached CPU, handling their stalls together with cache misses.

Note

You must write your own Verilog test benches to verify your designs. An example test trace will be provided, as well as a skeleton in `phase_6/skeletons/`.
You also should reference the RV32I reference card from webcourses for the instruction formats, opcodes, and immediate values.

1 Submission

- **One submission per group.** You will manually submit your work to Webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit**:

```
hart.v
alu.v
imm.v
rf.v
decoder.v
cache.v
fpu.v
additional verilog files (if any)
```

One module per file, and name the module the same as the filename. The testbenches instantiate modules named `hart`, `alu`, `imm`, `rf`, `decoder`, `cache` and `fpu` by name.

2 Grading Rubric

Category	Points
Multiply and divide	1 pts
FP move and compare	1 pts
FP convert	1 pts
FP arithmetic (add, multiply, fused)	1 pts
FP divide and square root	1 pts
FP control and status	1 pts
Mixed	1 pts
Stalls and cache misses	1 pts
Total	**8 pts**

Section 6.1 lists what each category tests, and Section **??** explains how each one is graded.

3 Verilog Coding Rules

Your submission is internally checked against a synthesizable subset. The rules listed below will result in a 0 for the specific file.

Rule	What it means for you
`*`, `/`, `%`, `**` only in constant expressions	Allowed in `parameter` and `localparam` definitions, in port and wire widths such as `[BLOCK_SIZE*8-1:0]`, and in generate indices. Not allowed on signals. The result must be computable at compile time.
No `===`, `!==`, `==?`, `!=?`	Not synthesizable. Use `==` and `!=`.
Shifts only by a literal	`x << 3` is fine; `x << i_op2[4:0]` and `x << SOME_PARAMETER` are both *rejected*. When the shift amount is a parameter, use a part-select or a concatenation instead.
No `if`/`else` in combinational logic	`if`/`else` is only legal inside a clocked `always @(posedge ...)` block. For combinational logic use a continuous `assign` with the `?:` operator, or a `case` statement inside `always @(*)`. A generate-level `if` on a parameter is fine.
No procedural loops	`for`, `while`, `repeat` and `do`/`while` statements are rejected. To build something repetitive, use a `generate` loop, or just write the instances out.
No delays or waits	`#5`, `wait`, `disable`, etc, are simulation constructs.
No system tasks	`$display`, `$finish`, `$stop`, `$system`, `$readmemh`/`$readmemb` and all file I/O (`$fopen`, `$fwrite`, ...) are not allowed in your submission. Only constant functions, i.e., `$clog2`, are allowed.
No `` `include ``, `` `ifdef ``, `` `ifndef ``, `` `elsif ``	Not needed. The only conditional directive allowed is the skeleton's `` `ifdef RISCV_FORMAL ``. This check happens if commented out.
Single clock, synchronous reset	`always @(posedge i_clk)` with `if (i_rst) ... else ...` inside it. No second signal in the sensitivity list. This applies to every pipeline register you add, not just the ones Phase 5 already had.

4 M and F extensions

Phase 6 introduces new ALU instructions for both integer and floating-point (FP) arithmetic, as well as a new FP datapath. The M extension adds integer multiply and divide. The F extension adds single-precision floating point, with 32 FP registers, a control and status register (fcsr), and multi-cycle divide and square root.

For more information on the M-extension, F-extension, and CSR instructions the RISC-V specification is available for you to reference.

- M extension: https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html
- F extension: https://docs.riscv.org/reference/isa/v20260120/unpriv/f-st-ext.html
- CSR instructions (Zicsr): https://docs.riscv.org/reference/isa/v20260120/unpriv/zicsr.html

This phase does not use the RV64-only instructions (mulw, divw, divuw, remw, remuw, fcvt.l[u].s and fcvt.s.l[u]), the Zmmul subset, the D and Q extensions, or anything privileged.

4.1 CPU datapath

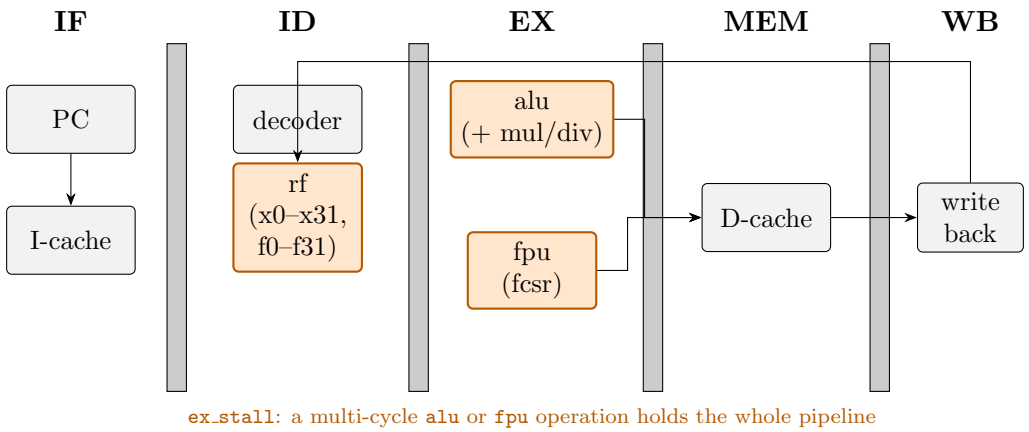


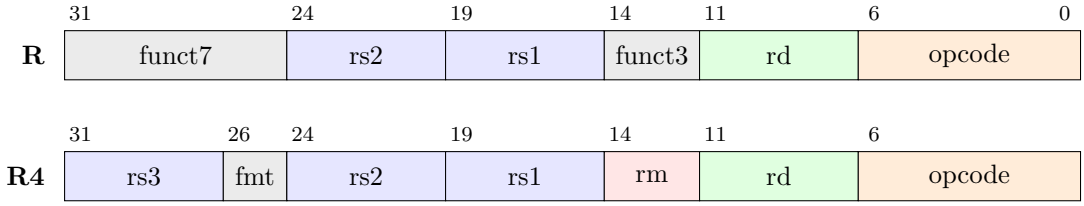
Figure 1: The phase 5 pipeline with the new and changed blocks (orange). The ALU adds multiply and divide, the register file adds the 32 FP registers, and the new `fpu` sits in the EX stage beside the ALU. Both caches are unchanged from Phase 5.

Figure 1 shows the Phase 5 pipeline with the new modules, and the table lists the change to each file.

File	Change
`imm.v`, `cache.v`	Unchanged from Phase 5.
`alu.v`	Adds integer multiply and divide (Section 5.1).
`rf.v`	Adds the 32 floating-point registers, with 3 read ports and a write enable (Section 5.2).
`fpu.v`	**New.** The floating-point unit and the `fcsr` register (Section 5.3).
`decoder.v`	Decodes the M, F and CSR instructions, and reports which register file each operand uses (Section 5.4).
`hart.v`	Instantiates the FPU, adds the FP register paths, the FP hazards and the stall for multi-cycle operations, and drives the new retire ports (Sections 5.5–5.7).

4.2 Instruction formats

M instructions use the R-type format. F instructions use R, I (flw) and S (fsw), plus one new format, R4, for the three-source fused instructions. R4 replaces the upper five bits of funct7 with rs3. The remaining two bits, fmt, select the precision and must be 00 (single precision).



In F instructions that round, funct3 is the rounding mode rm (Section 4.5). The new instructions use these opcodes:

Opcode	Name	Instructions
`0110011`	OP	RV32I register-register ops, and RV32M (`funct7 = 0000001`)
`0000111`	LOAD-FP	`flw` (`funct3 = 010`)
`0100111`	STORE-FP	`fsw` (`funct3 = 010`)
`1010011`	OP-FP	every other F instruction except the fused ones
`1000011`	MADD	`fmadd.s`
`1000111`	MSUB	`fmsub.s`
`1001011`	NMSUB	`fnmsub.s`
`1001111`	NMADD	`fnmadd.s`
`1110011`	SYSTEM	`ebreak`, and the CSR instructions (`funct3` $\neq$ `000`)

4.3 Integer multiplication and division (M extension)

The M extension adds eight R-type instructions on the OP opcode, distinguished from add and sub by funct7 = 0000001. funct3 selects the operation. [M spec §11.1]

Instruction	funct3	rd =
`mul`	000	Low 32 bits of `rs1` × `rs2`.
`mulh`	001	High 32 bits of **signed** `rs1` × **signed** `rs2`.
`mulhsu`	010	High 32 bits of **signed** `rs1` × **unsigned** `rs2`.
`mulhu`	011	High 32 bits of **unsigned** `rs1` × **unsigned** `rs2`.
`div`	100	Signed `rs1` / `rs2`, rounded toward zero.
`divu`	101	Unsigned `rs1` / `rs2`.
`rem`	110	Signed remainder. Takes the sign of `rs1`.
`remu`	111	Unsigned remainder.

4.3.1 Multiplication

A 32-bit multiply produces a 64-bit product. mul returns the lower 32 bits, and the three mulh instructions return the upper 32 bits. The lower half is the same for signed and unsigned operands, but the upper half is not, so mulh, mulhsu and mulhu differ in how they read the operands.

For example, with rs1 = rs2 = 0xFFFFFFFF (-1 signed, $2^{32}-1$ unsigned), all four instructions agree on the lower half, 0x00000001, but not on the upper half:

Instruction	Operands	64-bit product	rd
`mul`	any	`0x..._00000001`	`0x00000001`
`mulh`	-1×-1	`0x00000000_00000001`	`0x00000000`
`mulhsu`	$-1 \times (2^{32}-1)$	`0xFFFFFFFF_00000001`	`0xFFFFFFFF`
`mulhu`	$(2^{32}-1) \times (2^{32}-1)$	`0xFFFFFFFE_00000001`	`0xFFFFFFFE`

Binary multiplication is done using long multiplication (Figure 2). Each multiplier bit selects the multiplicand or zero, shifted to that bit's position, and the rows are summed.

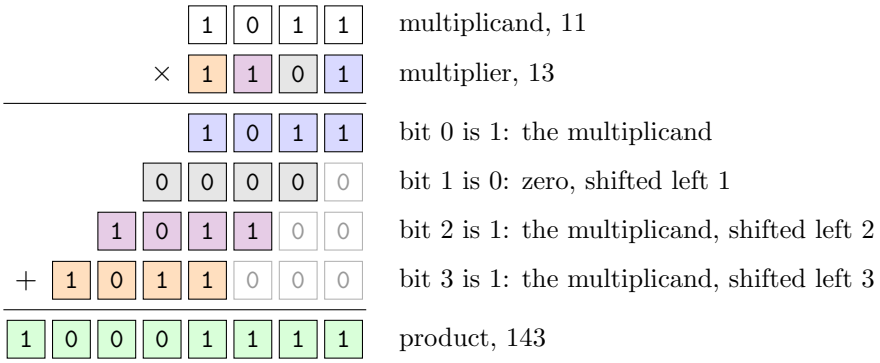


Figure 2: 11×13 with 4-bit operands. Each multiplier bit and the partial product it selects share a color. The gray zeros on the right come from the shift. The product of two 4-bit numbers needs 8 bits, just as two 32-bit numbers need 64.

To implement binary multiplication using hardware we can either sum up the partial products all at once or once per cycle. An iterative multiplier (Figure 3) adds one partial product per cycle. Each cycle it adds the multiplicand to the product if the lowest multiplier bit is 1, then shifts the multiplicand left and the multiplier right. An n-bit multiply takes n cycles

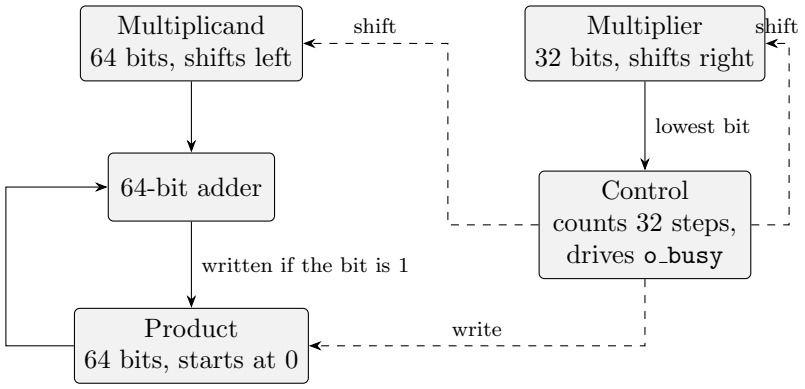


Figure 3: An iterative shift-and-add multiplier. Dashed lines are control signals.

The table traces 11×13 through the iterative multiplier. The bold bit selects add or no add.

Cycle	Multiplier	Multiplicand	Action	Product after
1	1101	00001011	add	00001011
2	0110	00010110	no add	00001011
3	0011	00101100	add	00110111
4	0001	01011000	add	10001111 (143)

The other way to sum partial products using hardware is with a combinational (array) multplier (Figure 4). Unlike the iterative multiplier, this design builds every partial product at once and sums them with a chain of adders. This results in a single-cycle multiply, at the cost of more area and a long logic path.

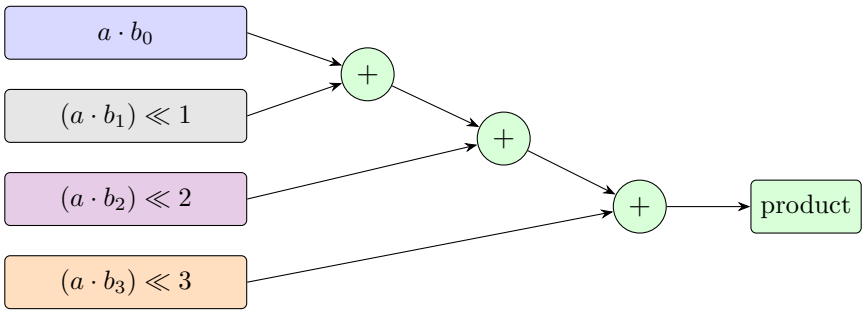


Figure 4: A combinational multiplier for 4-bit operands $a \times b$. $a \cdot b_i$ is every bit of a ANDed with bit i of b, and the shifts are by constants. A 32-bit multiplier has 32 partial products and 31 adders.

Both designs are unsigned. Signed multiplication multiplies the magnitudes and corrects the sign of the product (Figure 5).

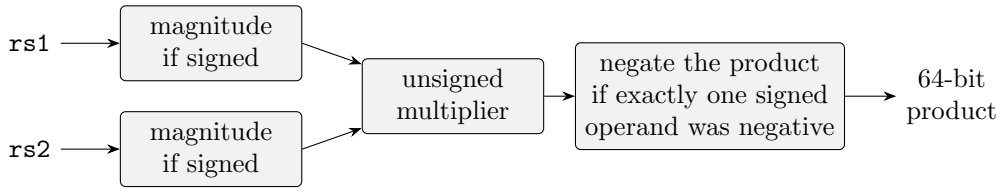


Figure 5: Signed multiplication from an unsigned multiplier. The magnitude of a negative number is its two's complement negation, and the 64-bit product is negated the same way.

Instruction	`rs1`	`rs2`	`rd` =
`mul`	either	either	product[31:0]
`mulh`	signed	signed	product[63:32]
`mulhsu`	signed	unsigned	product[63:32]
`mulhu`	unsigned	unsigned	product[63:32]

For example, mulhsu of -1 and 0xFFFFFFFF multiplies $1\times$ 0xFFFFFFFF = 0x00000000_FFFFFFFF. Only rs1 is negative, so the product is negated to 0xFFFFFFFF_00000001, matching the table above. The magnitude of -2^{31} fits only as an unsigned value (0x80000000).

4.3.2 Division

Division has two cases with no mathematical answer. One being division by zero, and the other being $-2^{31}/-1$, whose quotient 2^{31} does not fit in 32 signed bits. RISC-V division never traps, and each case has a defined result. [M spec §11.1.2, Table 1]

Case	`div`	`divu`	`rem`	`remu`
Divide by zero ($x/0$)	`0xFFFFFFFF` (-1)	`0xFFFFFFFF`	x	x
Overflow ($-2^{31}/-1$)	-2^{31} (`0x80000000`)	n/a	`0`	n/a

Signed division rounds the quotient toward zero, and the remainder takes the sign of the dividend, so rs1 = quotient $\times$ rs2 + remainder always holds. For example, $7/-3 = -2$ remainder 1, and $-7/3 = -2$ remainder -1.

Binary division is long division, called restoring division. Each step shifts the next dividend bit, from the top, into a partial remainder. If the remainder is at least the divisor, the divisor is subtracted and the quotient bit is 1. Otherwise the quotient bit is 0. The table divides 1101_2 (13) by 0011_2 (3):

Step	**Bit shifted in**	**Remainder**	**$\geq$ divisor?**	**Quotient so far**
1	`1`	`0001`	no	`0`
2	`1`	`0011`	yes, so `0011` $-$ `0011` = `0000`	`01`
3	`0`	`0000`	no	`010`
4	`1`	`0001`	no	`0100`

The result is quotient $0100_2 = 4$ and remainder 1. Like multiplication, division takes one step per cycle, n cycles for an n-bit value, or can be unrolled into n stages. fdiv.s uses the same algorithm (Section 5.3.2).

Signed division divides the magnitudes, negates the quotient if the signs differ, and gives the

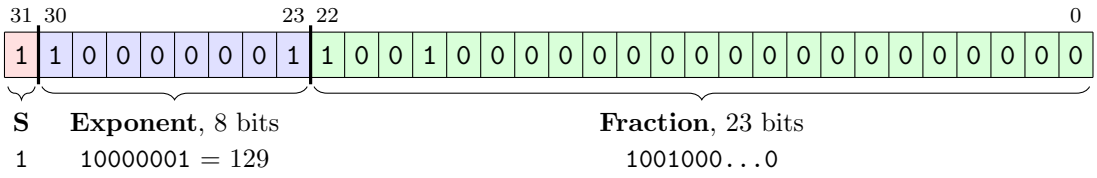
remainder the dividend's sign. The unsigned algorithm produces most special results directly: division by 0 gives an all-ones quotient with the dividend as the remainder, and the overflow case gives 0x80000000. The exception is div by zero with a negative dividend, where the sign correction gives 1 instead of −1.

4.4 Floating-point representation (IEEE 754)

4.4.1 Number format

A 32-bit integer cannot represent fractions such as 0.1, or values above about four billion. Floating point represents numbers in binary scientific notation, $\pm 1.f \times 2^e$. The exponent covers a large range, but the number of significand bits is fixed, so most results are rounded.

The F extension uses 32-bit single-precision numbers, defined by IEEE 754, with three fields:



Field	Bits	Meaning
Sign `S`	[31]	`0` positive, `1` negative. Zero has a sign too: +0 and −0 are different bit patterns.
Exponent `E`	[30:23]	Stored with a **bias** of 127: the real exponent is $E - 127$.
Fraction `F`	[22:0]	The bits after the binary point. The leading `1.` is not stored (the *hidden bit*).

For a normal number, the value is

$$(-1)^S \times 1.F \times 2^{E-127}$$

The leading 1 is implied and not stored, giving 24 bits of precision from 23 stored bits.

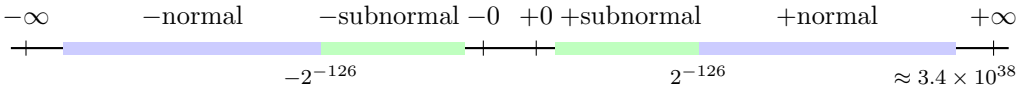
The figure encodes −6.25. The sign is 1, and $6.25 = 110.01_2 = 1.1001_2 \times 2^2$, so the exponent is $2 + 127 = 129$ and the fraction is 1001 followed by zeros. The result is 0xC0C80000.

4.4.2 Value classes

The minimum and maximum exponents encode special values. Every bit pattern belongs to exactly one class:

Class	Exponent	Fraction	Value
Zero	`0`	`0`	±0.
Subnormal	`0`	$\neq 0$	$(-1)^S \times 0.F \times 2^{-126}$. No hidden bit, and the exponent is −126, not −127.
Normal	1 to 254	any	$(-1)^S \times 1.F \times 2^{E-127}$.
Infinity	`255`	`0`	$\pm\infty$.
NaN	`255`	$\neq 0$	Not a Number. **Quiet** if fraction bit 22 is 1, **signaling** if it is 0.

Subnormals fill the gap between zero and 2^{-126}, so values approach zero gradually. A NaN (not a number) is the result of an invalid operation, such as 0/0. NaN is unordered: it is not less than, equal to, or greater than any value, including itself.



Common values:

Value	Bits	Value	Bits
+0	`0x00000000`	−0	`0x80000000`
1.0	`0x3F800000`	−6.25	`0xC0C80000`
0.1 (rounded)	`0x3DCCCCCD`	1/3 (rounded)	`0x3EAAAAAB`
Largest finite	`0x7F7FFFFF`	Smallest normal	`0x00800000`
Smallest subnormal	`0x00000001`	+∞	`0x7F800000`
Canonical NaN	`0x7FC00000`	A signaling NaN	`0x7F800001`

Every NaN produced by an arithmetic instruction is the canonical NaN, 0x7FC00000, regardless of the input NaN. Only flw, fsw, the moves and sign injection copy a NaN's bits unchanged. [F spec §20.1.3]

4.4.3 Special-value arithmetic

Special values follow the rules below, and some also raise an exception flag (Section 4.6.1).

Operation	Result	Flags
Any operation with a NaN operand	canonical NaN	NV if any operand is a signaling NaN
$\infty - \infty$, $\infty \times 0$, $0/0$, ∞/∞	canonical NaN	NV
$\sqrt{x}$ for $x < 0$	canonical NaN	NV
$\sqrt{-0}$	-0	none
$x/0$ for finite $x \neq 0$	$\pm\infty$	DZ
$x + (-x)$, exact zero result	+0 (−0 when rounding down)	none

4.5 Rounding

4.5.1 Rounding modes

Most results need more than 24 significand bits to be exact. For example, 1/3 repeats forever in binary. The exact result is rounded in the direction set by the rounding mode, which an instruction selects with its rm field. [F spec §20.1.2, Table 2]

`rm`	Name	Meaning
`000`	RNE	Round to nearest, ties to even. The default.
`001`	RTZ	Round toward zero (truncate).
`010`	RDN	Round down, toward $-\infty$.
`011`	RUP	Round up, toward $+\infty$.
`100`	RMM	Round to nearest, ties to max magnitude (away from zero).
`101`, `110`		Reserved. **The instruction is illegal and traps.**
`111`	DYN	Dynamic: use the mode stored in the `frm` CSR (Section 4.6). If `frm` holds 5, 6 or 7, the instruction traps.

4.5.2 Guard and sticky bits

Rounding requires three pieces of information. Those being, the guard bit (the first bit below the last kept bit), the sticky bit (the OR of all bits below the guard bit), and the sign. Guard and sticky give four cases:

Guard	Sticky	Discarded part	RNE does
0	0	exactly 0	nothing (exact)
0	1	less than half an ulp	truncate
1	0	exactly half (a tie)	round to the even neighbor
1	1	more than half	round up (in magnitude)

An ulp (unit in the last place) is the value of the last kept bit. If guard or sticky is 1, the result is inexact.

Breakdown

$1/3 = 0.010101\ldots_2 = 1.0101\ldots01\,|\,0101\ldots_2 \times 2^{-2}$. After the 24 kept bits, the guard bit is 1 and the sticky bit is 1, so the true value is more than halfway to the next number up. Each mode rounds it:

Mode	1/3	−1/3
RNE	`0x3EAAAAAB` (up)	`0xBEAAAAAB` (away from 0)
RTZ	`0x3EAAAAAA` (down)	`0xBEAAAAAA` (toward 0)
RDN	`0x3EAAAAAA` (down)	`0xBEAAAAAB` (toward $-\infty$)
RUP	`0x3EAAAAAB` (up)	`0xBEAAAAAA` (toward $+\infty$)
RMM	`0x3EAAAAAB` (up)	`0xBEAAAAAB` (away from 0)

Every one of these results is inexact, so every one raises the NX flag.

The two nearest modes differ only on an exact tie (guard 1, sticky 0). RNE picks the even neighbor, and RMM rounds away from zero. For example, RNE rounds 2.5 to 2 and 3.5 to 4, while RMM gives 3 and 4.

4.5.3 Overflow and underflow

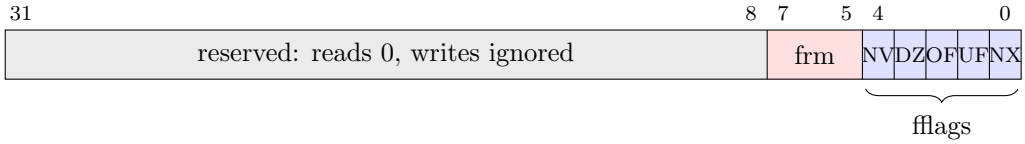
A result too large to represent overflows. Modes that round away from zero give infinity, and modes that round toward zero give the largest finite number:

Mode	Positive overflow	Negative overflow
RNE, RMM	$+\infty$ (`0x7F800000`)	$-\infty$ (`0xFF800000`)
RTZ	largest finite (`0x7F7FFFFF`)	$-$largest finite (`0xFF7FFFFF`)
RDN	largest finite (`0x7F7FFFFF`)	$-\infty$ (`0xFF800000`)
RUP	$+\infty$ (`0x7F800000`)	$-$largest finite (`0xFF7FFFFF`)

A result below 2^{-126} in magnitude is tiny and becomes a subnormal or zero: the significand is shifted right until the exponent is -126, with the shifted-out bits ORed into sticky, and then rounded. Subnormals are exact, never flushed to zero. Tininess is detected after rounding, so a value that rounds up to 2^{-126} does not raise UF. [F spec §20.1.4]

4.6 Floating-point control and status register (`fcsr`)

fcsr holds the dynamic rounding mode (frm) and the exception flags (fflags). Each field also has its own CSR address. [F spec §20.1.2]



CSR	Address	Width	Meaning
`fflags`	`0x001`	5	The accrued exception flags, `fcsr[4:0]`.
`frm`	`0x002`	3	The dynamic rounding mode, `fcsr[7:5]`.
`fcsr`	`0x003`	8	Both: {`frm, fflags`}. Bits [31:8] read as 0.

4.6.1 Exception flags

IEEE 754 defines five exceptions. In RISC-V they never trap: each sets a bit in fflags, and the instruction returns its default result. Flags are accrued, meaning they stay set until software clears them. [F spec §20.1.2, Table 3]

Bit	Flag	Raised when
4	NV (invalid)	The operation has no meaningful result: a signaling NaN operand, $\infty - \infty$, $0 \times \infty$, $0/0$, ∞/∞, $\sqrt{x < 0}$, a compare that signals on NaN, or a conversion to an integer that is out of range or NaN.
3	DZ (divide by zero)	A finite nonzero number is divided by zero.
2	OF (overflow)	The rounded result is too large (Section 4.5). OF always comes with NX.
1	UF (underflow)	The result is tiny **and** inexact. A tiny result that is exact does not raise UF.
0	NX (inexact)	The rounded result differs from the exact result.

For example, 1.0/3.0 raises NX, 1.0/0.0 raises DZ, 0.0/0.0 raises NV, and the largest finite number plus itself raises OF and NX. 0x00800000 $\times 0.5$ is an exact subnormal and raises nothing, while 0x00800001 $\times 0.5$ loses a bit and raises UF and NX.

4.6.2 CSR instructions

Six Zicsr instructions access fcsr. Each reads the old CSR value into rd, then writes the CSR. The i forms use the rs1 field as a 5-bit unsigned immediate. [Zicsr spec §5.1.1]

Instruction	funct3	Effect
`csrrw rd, csr, rs1`	001	`rd = csr; csr = rs1`
`csrrs rd, csr, rs1`	010	`rd = csr; csr = csr | rs1` (set bits). No write if `rs1` is `x0`.
`csrrc rd, csr, rs1`	011	`rd = csr; csr = csr & ~rs1` (clear bits). No write if `rs1` is `x0`.
`csrrwi rd, csr, uimm`	101	`rd = csr; csr = uimm`
`csrrsi rd, csr, uimm`	110	`rd = csr; csr = csr | uimm`. No write if `uimm` is 0.
`csrrci rd, csr, uimm`	111	`rd = csr; csr = csr & ~uimm`. No write if `uimm` is 0.

CSR instructions are I-type, with the CSR address in bits [31:20] and the SYSTEM opcode. With rd = x0, the CSR is still written. Pseudo-instructions such as frflags rd (csrrs rd, fflags, x0) are ordinary CSR instructions, listed in [F spec §20.1.2].

Breakdown

Instruction	`rd`	`fcsr` **after** (frm, fflags)
(start)		000, 00000
`fdiv.s f3, f1, f2` $(7/-3)$		000, 00001 (NX)
`csrrs x12, fflags, x0`	`x12 = 1`	000, 00001
`csrrwi x0, frm, 1`		001, 00001 (RTZ)
`fcvt.w.s x14, f3` (dynamic rm)	`x14 = -2`	001, 00001
`csrrw x5, fcsr, x0`	`x5 = 0x21`	000, 00000

$7/-3 = -2.333\ldots$, so the conversion rounds toward zero to -2 under RTZ. It would be -2 under RNE as well, but -3 under RDN. The last line reads fcsr (001_00001 = 0x21) and writes 0 to it in one instruction.

4.7 Floating-point instructions (F extension)

The F extension has 26 instructions. Operands named x are integer registers, and all others are FP registers f0–f31. In the tables, R marks a rounding-mode field, and a fixed value must match exactly or the instruction is illegal.

4.7.1 Loads and stores

flw and fsw are word loads and stores with an *integer* base register and an *FP* data register. They move bits unchanged and raise no flags. [F spec §20.1.5]

Instruction	Format	opcode	funct3	Effect
`flw fd, imm(xs1)`	I	`0000111`	`010`	`fd = mem[xs1 + imm]`
`fsw fs2, imm(xs1)`	S	`0100111`	`010`	`mem[xs1 + imm] = fs2`

4.7.2 Arithmetic

Every result is correctly rounded: the exact result, rounded once. Special operands follow Section 4.4. [F spec §20.1.6]

Instruction	funct7	rs2	rm	Effect
`fadd.s fd, fs1, fs2`	`0000000`	`fs2`	R	`fd = fs1 + fs2`
`fsub.s fd, fs1, fs2`	`0000100`	`fs2`	R	`fd = fs1 - fs2`
`fmul.s fd, fs1, fs2`	`0001000`	`fs2`	R	`fd = fs1` $\times$ `fs2`
`fdiv.s fd, fs1, fs2`	`0001100`	`fs2`	R	`fd = fs1 / fs2`
`fsqrt.s fd, fs1`	`0101100`	`00000`	R	`fd` = $\sqrt{\texttt{fs1}}$

4.7.3 Fused multiply-add

The fused instructions compute a multiply and an add with a single rounding at the end. This is more accurate than fmul.s followed by fadd.s, so the two can give different results. They are R4-type with fmt = 00. [F spec §20.1.6]

Instruction	opcode	Effect
`fmadd.s fd, fs1, fs2, fs3`	`1000011`	`fd =` $(\texttt{fs1} \times \texttt{fs2}) + \texttt{fs3}$
`fmsub.s fd, fs1, fs2, fs3`	`1000111`	`fd =` $(\texttt{fs1} \times \texttt{fs2}) - \texttt{fs3}$
`fnmsub.s fd, fs1, fs2, fs3`	`1001011`	`fd =` $-(\texttt{fs1} \times \texttt{fs2}) + \texttt{fs3}$
`fnmadd.s fd, fs1, fs2, fs3`	`1001111`	`fd =` $-(\texttt{fs1} \times \texttt{fs2}) - \texttt{fs3}$

The "n" negates the *product*: fnmsub.s is $-(a \times b) + c$. fnmadd.s is not a negated fmadd.s, since the two round differently under RDN and RUP and give different zero signs. $\infty \times 0$ raises NV even when fs3 is a quiet NaN.

4.7.4 Sign injection

Sign injection returns fs1's bits [30:0] with a new sign: fs2's sign, its inverse, or the XOR of both signs. Nothing is rounded and no flags are raised. With both sources the same, these implement fmv.s, fneg.s and fabs.s. [F spec §20.1.7]

Instruction	funct7	funct3	Result sign
`fsgnj.s fd, fs1, fs2`	0010000	000	sign of `fs2`
`fsgnjn.s fd, fs1, fs2`	0010000	001	NOT sign of `fs2`
`fsgnjx.s fd, fs1, fs2`	0010000	010	`fs1` XOR `fs2` signs

For example, with f1 = 7.0 and f2 = −3.0, fsgnj.s f3, f1, f2 gives −7.0 and fsgnjx.s f3, f2, f2 gives 3.0.

4.7.5 Compare and classify

These instructions do not round.

Instruction	funct7	funct3	Effect
`fmin.s fd, fs1, fs2`	0010100	000	`fd = min(fs1, fs2)`
`fmax.s fd, fs1, fs2`	0010100	001	`fd = max(fs1, fs2)`
`feq.s xd, fs1, fs2`	1010000	010	`xd = (fs1 == fs2)`
`flt.s xd, fs1, fs2`	1010000	001	`xd = (fs1 < fs2)`
`fle.s xd, fs1, fs2`	1010000	000	`xd = (fs1 <= fs2)`
`fclass.s xd, fs1`	1110000	001	`xd` = class of `fs1`; `rs2 = 00000`

Compares write 1 or 0 to an *integer* register. A compare with a NaN is false, and +0 equals −0. feq.s raises NV only for a signaling NaN, while flt.s and fle.s raise NV for any NaN. [F spec §20.1.8]

fmin.s and fmax.s treat −0 as less than +0. With one NaN operand they return the other operand, and with two, the canonical NaN. NV is raised only for a signaling NaN. [F spec §20.1.6]

fclass.s writes a one-hot mask of fs1's class to an integer register, and raises no flags. [F spec §20.1.9, Table 6]

`xd` bit	`fs1` is	`xd` bit	`fs1` is
0	$-\infty$	5	positive subnormal
1	negative normal	6	positive normal
2	negative subnormal	7	$+\infty$
3	−0	8	signaling NaN
4	+0	9	quiet NaN

4.7.6 Conversion and move

These instructions move values between the integer and FP register files.

Instruction	funct7	rs2	rm	Effect
`fcvt.w.s xd, fs1`	1100000	00000	R	float to signed integer
`fcvt.wu.s xd, fs1`	1100000	00001	R	float to unsigned integer
`fcvt.s.w fd, xs1`	1101000	00000	R	signed integer to float
`fcvt.s.wu fd, xs1`	1101000	00001	R	unsigned integer to float
`fmv.x.w xd, fs1`	1110000	00000	000	`xd = fs1` (bits copied)
`fmv.w.x fd, xs1`	1111000	00000	000	`fd = xs1` (bits copied)

Float-to-integer conversion rounds in the selected mode, then range-checks. A NaN or an out-of-range result is clipped and raises NV. A rounded in-range result raises NX, never together with NV. [F spec §20.1.7, Table 5]

Input	`fcvt.w.s`	`fcvt.wu.s`
NaN	`0x7FFFFFFF` ($2^{31}-1$), NV	`0xFFFFFFFF` ($2^{32}-1$), NV
$+\infty$, or rounds above the range	`0x7FFFFFFF`, NV	`0xFFFFFFFF`, NV
$-\infty$, or rounds below the range	`0x80000000` (-2^{31}), NV	`0x00000000`, NV
In range, not an integer	rounded value, NX	rounded value, NX
In range, an integer	exact value, no flags	exact value, no flags

Since the range check follows rounding, fcvt.wu.s of -0.3 gives 0 with NX under RNE, but 0 with NV under RDN, where it rounds to -1. Integer-to-float conversion rounds only above 24 significant bits: fcvt.s.w of $2^{24}+1$ gives 2^{24} and NX.

fmv.x.w and fmv.w.x copy bits unchanged and raise no flags, so fmv.x.w of 1.0 gives 0x3F800000, not 1.

5 Implementation

This section covers the hardware: the new modules, their pipeline integration, and the design requirements.

5.1 ALU

The ALU adds the eight M instructions. i_mext selects them, and i_opsel (funct3) then picks the operation, so the M result leaves on o_result like any other ALU result. Multiply and divide may be combinational or iterative. A combinational design finishes in one cycle and ties o_busy to 0. An iterative design raises o_busy until it is done, which stalls the pipeline (Section 5.5.2). It must hold o_result until i_advance, since a cache miss can keep the instruction in EX after the result is ready, and it must not restart while the pipeline is frozen. All RV32I operations stay combinational, with o_busy low.

Port	Dir/Width	Meaning
`i_clk`, `i_rst`	in	**New.** Clock, and synchronous reset.
`i_opsel`	in [2:0]	Operation (funct3). With `i_mext`, selects the M operation (Section 4.3).
`i_sub`	in	Subtract instead of add.
`i_unsigned`	in	Unsigned branch comparison.
`i_arith`	in	Arithmetic right shift.
`i_mext`	in	**New.** The instruction in EX is RV32M (decoder `o_mext`).
`i_advance`	in	**New.** The instruction in EX moves to MEM on the next rising edge.
`i_op1`	in [31:0]	First operand.
`i_op2`	in [31:0]	Second operand.
`o_result`	out [31:0]	The 32-bit result, including the M result.
`o_eq`, `o_slt`	out	Branch comparison results.
`o_busy`	out	**New.** An M operation is still in progress; the instruction must stay in EX.

Listing 1: The alu module header

```
module alu (
    input  wire        i_clk,
    input  wire        i_rst,
    input  wire [ 2:0] i_opsel,
    input  wire        i_sub,
    input  wire        i_unsigned,
    input  wire        i_arith,
    input  wire        i_mext,
    input  wire        i_advance,
    input  wire [31:0] i_op1,
    input  wire [31:0] i_op2,
    output wire [31:0] o_result,
    output wire        o_eq,
    output wire        o_slt,
    output wire        o_busy
);
```

5.2 Register file

rf holds both register sets: x0–x31 and f0–f31. The integer ports are unchanged from Phase 5. The FP side has three differences: f0 is an ordinary register (no hardwired zero), writes use an explicit write enable, and a third read port supplies fs3. BYPASS_EN applies to both sets.

Port	Dir/Width	Meaning
i_clk, i_rst	in	Clock, and synchronous reset. Reset also clears every FP register to +0.0.
i_rs1_raddr, o_rs1_rdata	in/out	Integer read port 1 (Phase 5).
i_rs2_raddr, o_rs2_rdata	in/out	Integer read port 2 (Phase 5).
i_rd_waddr, i_rd_wdata	in	Integer write port (Phase 5).
i_frs1_raddr	in [4:0]	**New.** FP read port 1 address.
o_frs1_rdata	out [31:0]	**New.** FP read port 1 data.
i_frs2_raddr	in [4:0]	**New.** FP read port 2 address.
o_frs2_rdata	out [31:0]	**New.** FP read port 2 data.
i_frs3_raddr	in [4:0]	**New.** FP read port 3 address.
o_frs3_rdata	out [31:0]	**New.** FP read port 3 data.
i_frd_wen	in	**New.** FP write enable.
i_frd_waddr	in [4:0]	**New.** FP write address.
i_frd_wdata	in [31:0]	**New.** FP write data.

Listing 2: The rf module header

```
module rf #(
    parameter BYPASS_EN = 0
) (
    input  wire        i_clk,
    input  wire        i_rst,
    // integer registers (unchanged from Phase 5)
    input  wire [ 4:0] i_rs1_raddr,
    output wire [31:0] o_rs1_rdata,
    input  wire [ 4:0] i_rs2_raddr,
    output wire [31:0] o_rs2_rdata,
    input  wire [ 4:0] i_rd_waddr,
    input  wire [31:0] i_rd_wdata,
    // floating-point registers
    input  wire [ 4:0] i_frs1_raddr,
    output wire [31:0] o_frs1_rdata,
    input  wire [ 4:0] i_frs2_raddr,
    output wire [31:0] o_frs2_rdata,
    input  wire [ 4:0] i_frs3_raddr,
    output wire [31:0] o_frs3_rdata,
    input  wire        i_frd_wen,
    input  wire [ 4:0] i_frd_waddr,
    input  wire [31:0] i_frd_wdata
);
```

Reads are combinational and writes are synchronous. With BYPASS_EN = 1, a write in flight is always bypassed to any read port addressing the same register of the same set, f0 included.

5.3 Floating-point unit

fpu sits in EX beside the ALU and owns fcsr. It executes every OP-FP, fused and CSR instruction, including the integer/float conversions and moves. flw and fsw do not use it. It decodes funct7, rs2, rm and funct3 from the instruction word.

Port	Dir/Width	Meaning
`i_clk`, `i_rst`	in	Clock, and synchronous reset (clears `fcsr`).
`i_inst`	in [31:0]	The instruction word in EX.
`i_valid`	in	`i_inst` is a legal FPU instruction (decoder `o_fpu`).
`i_advance`	in	The instruction in EX moves to MEM on the next rising edge.
`i_frs1`, `i_frs2`, `i_frs3`	in [31:0]	FP source operands, raw bit patterns.
`i_xrs1`	in [31:0]	Integer `rs1`, forwarded, for `fmv.w.x`, `fcvt.s.w[u]` and the CSR register forms.
`o_fp_result`	out [31:0]	Result for an instruction that writes an FP register.
`o_int_result`	out [31:0]	Result for an instruction that writes an integer register, including the old CSR value.
`o_fflags`	out [4:0]	The accrued flags as they will be after this instruction (for the retire trace).
`o_busy`	out	A multi-cycle operation is in progress; the instruction must stay in EX.
`o_rm_trap`	out	Dynamic `rm` with a reserved `frm` value: the instruction traps.

Listing 3: The fpu module header

```
module fpu (
    input  wire        i_clk,
    input  wire        i_rst,
    input  wire [31:0] i_inst,
    input  wire        i_valid,
    input  wire        i_advance,
    input  wire [31:0] i_frs1,
    input  wire [31:0] i_frs2,
    input  wire [31:0] i_frs3,
    input  wire [31:0] i_xrs1,
    output wire [31:0] o_fp_result,
    output wire [31:0] o_int_result,
    output wire [ 4:0] o_fflags,
    output wire        o_busy,
    output wire        o_rm_trap
);
```

5.3.1 FPU organization

The internal organization is up to you. A common first stage unpacks each operand into sign, exponent and significand and classifies it. Giving a subnormal exponent 1 and hidden bit 0 lets it be treated as a normal number. Sign injection, min/max, compares, class and moves need no rounding. All other operations compute an exact result and then round it.

Note

Seven instructions need a multiply, an add, or both: fadd, fsub, fmul and the four fused instructions. One multiply-add datapath, $a \times b + c$, can serve all seven, since fadd is $a \times 1.0 + b$. Likewise, one rounder can serve every rounding instruction, and one multiplier design can serve both the FPU and the ALU.

The multiply-add needs a 24×24-bit significand product, built like the integer multiplier (Section 4.3). An iterative multiplier stalls these instructions through o_busy. fmul must skip the addend rather than add $+0$: $-0 \times 5 + 0$ gives $+0$, but fmul must return -0.

Aligning the addend and normalizing the result need variable shifts, which the style rules forbid (Section 3). Barrel shifters built from constant-shift stages, as in the Phase 2 ALU, provide them. On right shifts, the shifted-out bits are ORed into sticky.

5.3.2 Divide and square root

fdiv.s and fsqrt.s use restoring division (Section 4.3), and square root consumes two radicand bits per step. About 26 steps produce 24 bits plus a guard bit, and a nonzero remainder sets sticky. Special operands can return immediately.

Latency is up to you, since cycles are not graded. While an operation runs, the hart holds every instruction in place (Section 5.5.2).

5.3.3 `fcsr` updates

On reset, fcsr is 0: no flags set, and frm is RNE. The FPU ORs each instruction's flags into fflags and never overwrites them. Every F instruction reports the accrued flags at retire (Section 5.7), so an overwrite appears in the trace on the next instruction.

fcsr changes only once per instruction, on the edge where the instruction leaves EX (i_advance high). Writing it on every cycle in EX breaks CSR instructions held by a cache miss: csrrw x5, fcsr, x6 would read back x6 instead of the old value.

5.4 Decoder

The decoder accepts every M, F and CSR encoding in this handout, rejects all others, and reports which register file each operand uses. F instructions use integer registers through the operands named x in Section 4.7 and through the CSR instructions. fflags, frm and fcsr are the only CSRs, so any other CSR address is illegal. The FPU decodes its own operation, so the decoder adds only the outputs below.

Port	Width	Meaning
`o_rd_sel`	[4:0]	**Widened from 4 bits.** One-hot writeback select, one new source (table below).
`o_mext`	1	The instruction is RV32M (drives the ALU's `i_mext`).
`o_fpu`	1	The instruction is executed by the FPU: every OP-FP and fused instruction, and the CSR instructions. Not `flw`/`fsw`.
`o_frs1_en`	1	The instruction reads FP `rs1`.
`o_frs2_en`	1	The instruction reads FP `rs2`. Includes `fsw`'s data register.
`o_frs3_en`	1	The instruction reads FP `rs3` (fused only).
`o_rs3`	[4:0]	`i_inst[31:27]`.
`o_frd_en`	1	The instruction writes FP register `rd`. `o_rd` is then `5'd0`.

Bit	Value	Source	Instructions
[0]	`5'b00001`	ALU result	OP (including RV32M), OP-IMM, `auipc`
[1]	`5'b00010`	immediate	`lui`
[2]	`5'b00100`	PC + 4	`jal`, `jalr`
[3]	`5'b01000`	memory	LOAD
[4]	`5'b10000`	FPU integer result	`fmv.x.w`, `fclass.s`, compares, `fcvt.w[u].s`, CSR instructions

For RV32M, o_alu_opsel carries funct3 and o_mext selects the M operations. flw and fsw drive the memory outputs like lw and sw.

5.4.1 Instruction legality

An instruction is legal only if every field matches an encoding above:

- OP with funct7 = 0000001 is RV32M, with any funct3. Any other funct7 besides 0000000/0100000 is still illegal.
- flw/fsw: only funct3 = 010.
- OP-FP: only the funct7/funct3/rs2 combinations in Section 4.7. Fixed fields (rs2 of fsqrt, the conversions and moves; funct3 of the non-rounding instructions) must match exactly.
- Fused: fmt (bits [26:25]) must be 00.
- A static rm of 101 or 110 is illegal.
- CSR instructions (SYSTEM, funct3 $\neq$ 000, not 100): only addresses 0x001, 0x002 and 0x003.

5.5 Pipeline integration

Each instruction class uses the five stages as follows:

Class	ID reads	EX	MEM	WB writes
RV32M	`rs1`, `rs2`	`alu`	pass	`rd` (integer)
FP arithmetic	`fs1`, `fs2` (`fs3`)	`fpu`	pass	`fd`
FP compare, class, `fmv.x.w`, `fcvt.w[u].s`	`fs1` (`fs2`)	`fpu`	pass	`rd` (integer)
`fcvt.s.w[u]`, `fmv.w.x`	`rs1`	`fpu`	pass	`fd`
CSR instructions	`rs1` (register forms)	`fpu`: read and write `fcsr`	pass	`rd` (old value)
`flw`	`rs1`	address (ALU)	D-cache read	`fd` (loaded word)
`fsw`	`rs1`, `fs2`	address (ALU)	D-cache write of `fs2`	nothing

5.5.1 Data hazards

Every Phase 4 hazard still applies, within each register file and across them. x5 and f5 are different registers, so hazard checks compare register numbers only within the same file. Ignoring the file causes false stalls, and checking only integer registers misses FP hazards.

Hazard	Example	Handling
M RAW	`mul x7, x5, x6` `add x8, x7, x0`	Like an ALU result: $EX \rightarrow EX$ forwarding.
Integer into FP	`addi x5, x0, 7` `fcvt.s.w f1, x5`	The FPU's `i_xrs1` is forwarded like any integer operand.
FP into integer	`feq.s x5, f1, f2` `add x6, x5, x0`	The FPU integer result forwards $EX \rightarrow EX$ like an ALU result (`o_rd_sel[4]`).
FP RAW	`fadd.s f3, f1, f2` `fmul.s f4, f3, f3`	**Hazard.** Stall until the writer reaches WB, where the `rf` bypass delivers it.
FP load-use	`flw f1, 0(x5)` `fadd.s f2, f1, f1`	**Hazard.** Stall, as above.
FP load-store	`fadd.s f1, f2, f3` `fsw f1, 0(x5)`	**Hazard** on `fsw`'s *data* register `fs2`.
Third operand	`fadd.s f3, f1, f2` `fmadd.s f4, f1, f2, f3`	**Hazard.** Check `rs3` too, not only `rs1`/`rs2`.
Rounding mode	`csrrwi x0, frm, 1` `fadd.s f3, f1, f2`	Must use the new mode. Handled if `fcsr` is written as the CSR instruction leaves EX.
Flags	`fdiv.s f3, f1, f2` `frflags x5`	Must see the new flag. Handled the same way.

Stalling resolves all FP-register hazards, and the reference forwards only integer results. FP forwarding is optional, since cycles are not graded.

5.5.2 Multi-cycle stall

A multi-cycle operation stays in EX until it finishes. The simplest approach freezes the whole pipeline, like a cache miss: pipe_stall = freeze | ex_stall, where ex_stall is high while the ALU or the FPU is busy. Every pipeline register, the PC and the retire logic hold on pipe_stall. An n-cycle operation proceeds as follows:

Cycle	Unit	CPU
1	Instruction enters EX, starts	Everything frozen
$2 - n$	One bit per cycle	Everything frozen
$n + 1$	Done, result held	Instruction advances to MEM
$n + 2$	Ready for the next instruction	Running

Every stage must freeze. Draining MEM and WB loses values the stalled instruction may need to forward, and an IF/ID register that keeps updating drops the next instruction.

5.5.3 Stalls and cache misses

The multi-cycle stall and cache misses can overlap. Three rules apply:

- No data cache access during the stall. The load or store in MEM waits and accesses the cache once. The request is gated with ˜ex_stall; otherwise a store is written on every frozen cycle.
- A finished result is held until the instruction advances, since the load or store ahead of it can only miss after the operation is done.
- The operation continues during a cache freeze. An instruction cache miss must not restart it.

The timeline below, measured on the reference, has lw x8, 0x400(x5) missing with fdiv.s f3, f1, f2 right behind it (26-cycle divide, memory delay 4). A multi-cycle mul or div behaves the same way.

Cycle	`lw` (MEM)	`fdiv.s` (EX)	Stall
1	waiting, no cache access	enters EX, starts	`ex_stall`
2–26	waiting, no cache access	one bit per cycle	`ex_stall`
27	cache access, **miss**	done, result held	`freeze`
28–30	line read in flight	result held	`freeze`
31	line arrives, load done	result held	none
32	`lw` retires	moves to MEM	none
33		`fdiv.s` retires	none

The load accesses memory once, and the divide result survives the freeze.

5.6 Memory regions

Instructions still start at 0x00400000 and data at 0x10010000, as in Phase 4. The cache works on full 32-bit addresses, so data addresses need no special handling: the high bits simply become part of the tag. `flw` and `fsw` use the same data memory as `lw` and `sw`.

5.7 Retire interface

These outputs are not part of RV32I. They exist for the testbench to validate your design and its functionality. Set each signal appropriately every cycle. As in Phase 4, `o_retire_valid` is not guaranteed high every cycle. Cache stalls and multi-cycle operations leave it low for as long as the pipeline is frozen, and that is not a defect.

Port	Width	Meaning
`o_retire_valid`	1	An instruction retired this cycle.
`o_retire_inst`	[31:0]	The raw fetched instruction word.
`o_retire_trap`	1	Illegal encoding, or a misaligned access.
`o_retire_halt`	1	The instruction is `ebreak`.
`o_retire_rs1_raddr`	[4:0]	Source register 1, `5'd0` **if not read**.
`o_retire_rs1_rdata`	[31:0]	The architecturally correct value `rs1` used, reflecting any forwarding.
`o_retire_rs2_raddr`	[4:0]	Source register 2, `5'd0` **if not read**.
`o_retire_rs2_rdata`	[31:0]	The architecturally correct value `rs2` used, reflecting any forwarding.
`o_retire_rd_waddr`	[4:0]	Destination register, `5'd0` **if no register is written**.
`o_retire_rd_wdata`	[31:0]	Value written to `rd`.
`o_retire_pc`	[31:0]	PC this instruction was fetched from.
`o_retire_next_pc`	[31:0]	`PC + 4`, or the branch or jump target when taken.
The retiring instruction's own data memory access, as presented to the data cache, timed to the writeback cycle:		
`o_retire_dmem_addr`	[31:0]	Word-aligned address of the load or store.
`o_retire_dmem_ren`	1	The instruction was a load. Never high with `o_retire_dmem_wen`.
`o_retire_dmem_wen`	1	The instruction was a store.
`o_retire_dmem_mask`	[3:0]	One bit per byte lane of the word, as in Phase 4.
`o_retire_dmem_wdata`	[31:0]	Store data, in place within the word. Only the masked lanes matter.
`o_retire_dmem_rdata`	[31:0]	The full word the data cache returned for a load.
New for Phase 6: the retiring instruction's floating-point registers and flags:		
`o_retire_frs1_raddr`	[4:0]	FP source register 1 (the `rs1` field), for instructions that read FP `rs1`.
`o_retire_frs1_rdata`	[31:0]	The value FP `rs1` had when the instruction read it.
`o_retire_frs2_raddr`	[4:0]	FP source register 2 (the `rs2` field), including `fsw`'s data register.
`o_retire_frs2_rdata`	[31:0]	The value FP `rs2` had when the instruction read it.
`o_retire_frs3_raddr`	[4:0]	FP source register 3 (the `rs3` field), fused instructions only.
`o_retire_frs3_rdata`	[31:0]	The value FP `rs3` had when the instruction read it.
`o_retire_frd_wen`	1	The instruction writes an FP register.
`o_retire_frd_waddr`	[4:0]	FP destination register.
`o_retire_frd_wdata`	[31:0]	Value written to the FP destination register.
`o_retire_fflags`	[4:0]	The accrued `fflags` (NV DZ OF UF NX) **after** this instruction.

Note

The testbench uses `o_retire_valid` and `o_retire_halt` to detect the final `ebreak`. Every port in this table must still exist with these exact names and widths or your hart will not compile against the testbench, and keep every value correct as in Phase 4: the retire trace is how your design is graded in this phase (Section **??**).

FP source fields are checked only for the FP operands an instruction reads. Integer operands of F instructions appear on o_retire_rs1_* and o_retire_rd_*. o_retire_fflags is checked on every F instruction, including flw, fsw and the CSR instructions, even when the flags are unchanged.

5.7.1 Traps

Traps work exactly as in Phase 4. A trapped instruction retires with `o_retire_trap` set, execution continues at pc+4, and the instruction has no side effects. For a misaligned load or store this now means it must not reach the cache at all, meaning no lookup, no miss, no line read and no write to main memory.

Phase 6 adds the traps below. A trapped instruction writes no register and leaves fcsr unchanged. Floating-point exceptions and division by zero are not traps.

- Any encoding Section 5.4 does not list as legal, including a static rm of 101 or 110 and any other CSR address.
- A dynamic rm while frm holds 101, 110 or 111. The FPU raises this trap in EX (o_rm_trap), since only it knows frm.
- A misaligned flw or fsw.

5.8 Design requirements

The tables below sum up what your design must do and which choices are yours.

Required	Detail
Files	The six Phase 5 files plus `fpu.v`, with these module names. Multiply and divide are in `alu.v`, and the FP registers are in `rf.v`.
RV32M	All 8 instructions, with the divide-by-zero and overflow results of Section 4.3, on a multiplier and divider you build.
RV32F	All 26 instructions, all 5 rounding modes and dynamic rounding, subnormals handled exactly, tininess after rounding, canonical NaN results.
`fcsr`	`fflags`, `frm` and `fcsr` through all six CSR instructions; flags accrue; changes only when the instruction leaves EX.
FP registers	In `rf.v`: 32 registers, 3 read ports, write enable, `f0` writable, same-cycle bypass.
Hazards	Every hazard in Section 5.5.1, in both register files and between them.
Multi-cycle stall	The whole pipeline holds while a multi-cycle ALU or FPU operation runs, and the D-cache does not repeat an access during it.
Caches	Your Phase 5 caches keep working with every parameter the testbench sets.
Retire interface	Every port in Section 5.7, correct for every retiring instruction.
Operators	No `*`, `/` or `%` on signals in any file, `alu.v` and `fpu.v` included (Section 3).

Design decision	Detail
Multiplier and divider design	Whether each multiplier and divider is combinational or iterative, how many cycles each instruction takes, and whether special operands finish early.
FPU organization	How you split the FPU into submodules (they may live in `fpu.v` or in additional files).
FP forwarding	Whether FP results are forwarded or only stalled on.
Hazard detection	Where in the pipeline you detect FP hazards and how you insert the bubbles.
Rounder design	How you represent intermediate values (exponent width, guard and sticky bits).

6 Testing Your Work

You must write your own testbenches. Two trace files are provided for one test, in the `traces/vectors/` directory.

Instruction memory file	Final data memory file
`random_program.hex`	`random_dmem.hex`

6.1 Trace files

`random_program.hex` is the initial instruction memory. It has one 32-bit word per line, the first word at address 0x00400000. It holds about 32000 random RV32IMF instructions in nine sections, ending with an `ebreak`. Initialize your data memory to all 0's.

`random_dmem.hex` is the final data memory. It has 32768 words, one per line, starting at address 0x10010000.

M and F results are in registers, which a memory image cannot show. About every 40 instructions, the program stores the registers it changed, plus fflags, to a register dump range, and each section ends with a full dump. Each section has its own range (table below). Program data is at 0x10010000–0x10017FFF, and a final series of loads from 0x10018000–0x1001FFFF flushes every dirty line, as in Phase 5.

Section	Tests	Dump range
muldiv	RV32M	`0x10020000–0x100217FF`
fp_move_compare	moves, sign injection, min/max, compares, class	`0x10021800–0x10022FFF`
fp_convert	conversions	`0x10023000–0x100247FF`
fp_add_mul	`fadd`, `fsub`, `fmul`	`0x10024800–0x10025FFF`
fp_fused	fused multiply-add	`0x10026000–0x100277FF`
fp_div_sqrt	`fdiv`, `fsqrt` and their stall	`0x10027800–0x10028FFF`
fp_csr	`fflags`, `frm`, `fcsr`, dynamic rounding	`0x10029000–0x1002A7FF`
mixed	all of the above	`0x1002A800–0x1002BFFF`
cache_stall	long operations next to cache misses	`0x1002C000–0x1002D7FF`

Each section matches one graded category, except fp_add_mul and fp_fused, which together form the FP arithmetic category.

scripts/student_test.sh runs the program in the graded cache configuration (PLRU, 4 ways, 2 KB, 16-byte blocks, prefetch on, 4-cycle memory) and compares the final memory with random_dmem.hex. A wrong word at 0x10020000 or above is in a register dump, and the table gives its section. This check is weaker than grading, since a wrong value overwritten before the next dump can pass. Other configurations work with -P, for BLOCK_SIZE $\geq$ 16 bytes and CACHE_SIZE $\leq$ 16 KB.

6.2 Setup check

Before worrying about correctness, confirm your design can be *run* by our grading setup at all. `scripts/check_design.sh` compiles your files exactly the way grading does, runs a 10-instruction program, and reports whether your `hart` instantiates, resets, retires instructions and halts:

```
./scripts/check_design.sh mine /path/to/your/verilog
Your design compiled successfully.
```

If something would stop grading, it instead prints one `ERROR:` line per problem, followed by the evidence (the offending lines, or the compiler's messages).

It does not check whether your processor is correct, that is what the traces above are for. It exists because a working processor can be scored as broken for reasons unrelated to its logic. The most common is **debug output left in your RTL**: grading reads the trace your processor produces *as text*, so anything your own code prints with `$display`, `$write` or `$monitor` is interleaved into that text and corrupts it, making instructions *disappear* from the trace we see. Comparing against the trace files above cannot warn you about this, because that comparison happens on signals inside Verilog and never parses text.

It also catches a `$finish` or `$stop` in your design (which ends the simulation early, so everything after it is never seen), `` `include `` directives, and anything that stops your files compiling together. Run it before you submit.

7 Learnings Overview

These are the learnings you will gain from this phase:

- **Integer multiply and divide.** Building a multiplier from shifts and adds and a divider from shifts and subtracts, signed and unsigned 64-bit products, and division that never traps: divide by zero and overflow have defined results.
- **IEEE 754 representation.** Sign, biased exponent and fraction; the hidden bit; normal and subnormal numbers, signed zeros, infinities, and quiet and signaling NaNs.
- **Rounding.** Guard and sticky bits, the five rounding modes, ties, overflow per mode, and tininess detected after rounding.
- **Exception flags.** The five IEEE exceptions, why they set flags instead of trapping, and how flags accrue until software clears them.
- **Control and status registers.** `fcsr`, `frm` and `fflags`, the six Zicsr instructions, and dynamic rounding.
- **Floating-point datapaths.** Unpacking, the fused multiply-add, alignment and normalization with barrel shifters, conversions, and bit-serial divide and square root.
- **A second register file.** Three read ports, a writable `f0`, and hazards within and across two register files.
- **Multi-cycle functional units.** Freezing a pipeline for a long operation, combining that stall with cache misses, and committing side effects exactly once.